



Hochschule für Technik
und Wirtschaft Berlin

University of Applied Sciences

Gehörtrainer Android App

Semesterbegleitende Projektarbeit
Mobile Betriebssysteme und Netzwerke

Name des Studiengangs

Angewandte Informatik (B)

Fachbereich 4

vorgelegt von

Alexander Schmidt 587471

Dozent: Prof. Dr.-Ing. Thomas Schwotzer

Sommersemester 2026

1 Gehörtrainer Android App.....	3
2 Use-Cases:.....	3
2.1 Use Case Diagramm.....	7
3 UI Screenshots.....	8
3.1 Hauptmenü.....	8
3.2 Einstellungen.....	9
3.3 Raten.....	10
3.4 Ranking.....	11
3.5 Filter.....	12
4 Komponenten.....	13
4.1 Komponentendiagramm.....	13
4.2 MVC.....	14
4.2.1 Model.....	14
4.2.2 View.....	14
4.2.3 Controller.....	14
4.3 Spiellogik.....	14
4.3.1 ISpiellogik.....	15
4.3.2 Spiellogik.....	15
4.3.3 Tests.....	15
4.4 AudioDevice.....	16
4.4.1 IAudioDevice.....	16
4.4.2 SoundpoolDevice.....	16
4.4.3 Tests.....	16
4.5 Spielergebnisse.....	16
4.5.1 ISpielergebnisRepository.....	16
4.5.2 SpielergebnisRepository.....	16
4.5.3 Tests.....	17
4.6 PreferencesRepository.....	17
4.6.1 IPreferencesRepository.....	17
4.6.2 PreferencesRepository.....	17
4.6.3 Tests.....	17
4.7 InstallationIdManager.....	18
4.7.1 Tests.....	18
4.8 Bluetooth.....	18
4.8.1 IBluetoothManager.....	18
4.8.2 BluetoothSyncController.....	18
4.8.3 Tests.....	19
4.9 UI.....	19
4.9.1 Tests.....	19
5 Rückblick auf das Projekt.....	20
6 Repository und Videopräsentation.....	20
7 Beschreibung Anwendungsablauf.....	21
7.1 Hauptmenü.....	21
7.2 Einstellungen.....	21
7.3 Raten.....	21
7.4 Bestenliste.....	22
7.5 Filter.....	22

1 Gehörtrainer Android App

Eine Android App für Musiker, die ihr Gehör für westliche Musik trainieren wollen.

Die Anwendung spielt einen Grundton und anschließend einen oder mehrere Töne. Der Anwender kann dann tippen, wie weit das Intervall zwischen den Tönen ist und bekommt dafür Feedback richtig/falsch.

Ergebnisse der Rateversuche werden lokal gespeichert. Ergebnisse können mit Nutzern via Bluetooth ausgetauscht werden, sodass eine lokal gespeicherte Bestenliste mit verschiedenen Nutzern entsteht.

2 Use-Cases:

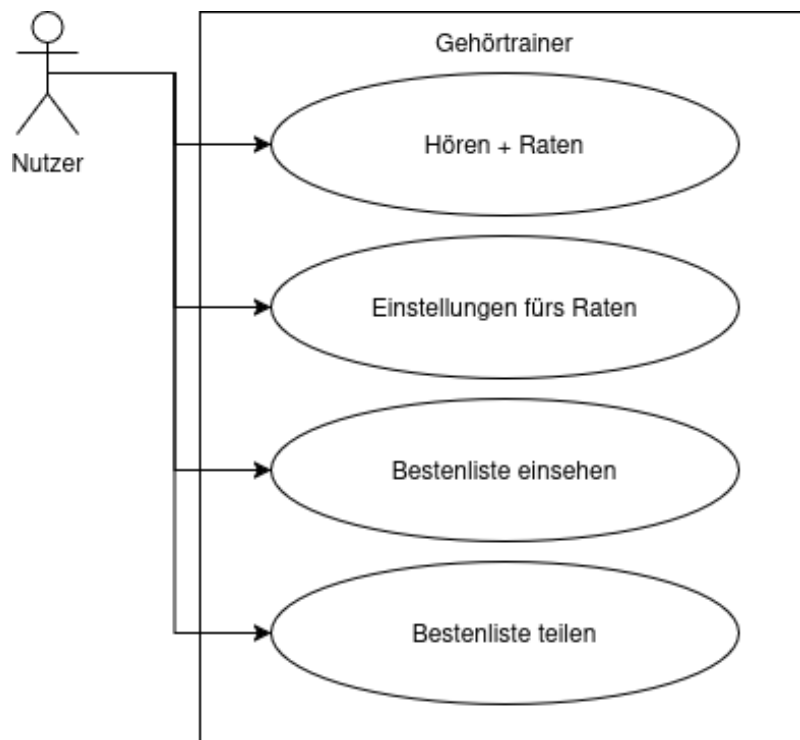
Name	UC-Hören/Raten 1 - Einstellungen vor Raten
Scope	Gehörtrainer App
Niveau	Anwenderziel
Actor	Nutzer (will raten)
Stakeholder	Nutzer (will raten)
Vorbedingung	- Anwendung gestartet
Mindestzusicherungen	- UI erlaubt speichern (und damit Ratestart) - UI erlaubt abbrechen ohne speichern
Zusicherung im Erfolgsfall	- Schwierigkeit des Ratens direkt abhängig von gewählten Einstellungen - UI wird aktualisiert für Ratemodus
Standardablauf	1. Anwender wählt 'Start' im Hauptmenü 2. Menü Schwierigkeit öffnet sich 3. Anwender wählt Schwierigkeit 4. Anwender wählt 'Start' in Einstellungsmenü 5. Einstellungen werden gespeichert 6. Übergang zu UC-Hören/Raten 2
Erweiterungen	4a: Anwender wählt 'Abbruch' -> ohne Speichern zu Hauptmenü

Name	UC-Hören/Raten 2 - Raten
Scope	Gehörtrainer App
Niveau	Anwenderziel
Actor	Nutzer (will raten)
Stakeholder	Nutzer (will raten)
Vorbedingung	- UC-Hören/Raten 1 :: Standardablauf
Mindestzusicherungen	- UI Raten wird angezeigt - Grundton und mind 1 weiterer Ton werden abgespielt - Nutzer kann Antwort eingeben
Zusicherung im Erfolgsfall	- Nutzer erhält Feedback fürs Raten - Wertung wird gespeichert
Standardablauf	1. Grundton wird abgespielt 2. Intervall/Akkord wird abgespielt 3. UI gibt Eingabe frei 4. Anwender gibt geratene(n) Wert(e) ein 5. Abschließende Bewertung gespeichert 6. Abschließende Bewertung angezeigt 7. Übergang zu Hauptmenü
Erweiterungen	1 bis 4: n-mal wiederholt falls in Einstellungen mehrere Rateversuche gewählt wurden Abbruch vor 5: Übergang zu Hauptmenü

Name	UC-Bestenliste 1 - Ansehen
Scope	Gehörtrainer App
Niveau	Anwenderziel
Actor	Nutzer (will Wertungen sehen)
Stakeholder	Nutzer (will Wertungen sehen)
Vorbedingung	- Anwendung gestartet
Mindestzusicherungen	- UI zeigt eine Bestenliste - UI erlaubt Rückkehr zum Hauptmenü
Zusicherung im Erfolgsfall	- Bestenliste kann beliebig nach Ratekategorien sortiert werden
Standardablauf	1. Anwender wählt 'Bestenliste' im Hauptmenü 2. UI Übergang zur ungefilterten Bestenliste 3. Anwender wählt nach beliebigen Filter um gespeicherte Ergebnisse anzuzeigen
Erweiterungen	3a: Rückkehr zum Hauptmenü via Button 3b: Austausch Button -> UC-Bestenliste 2

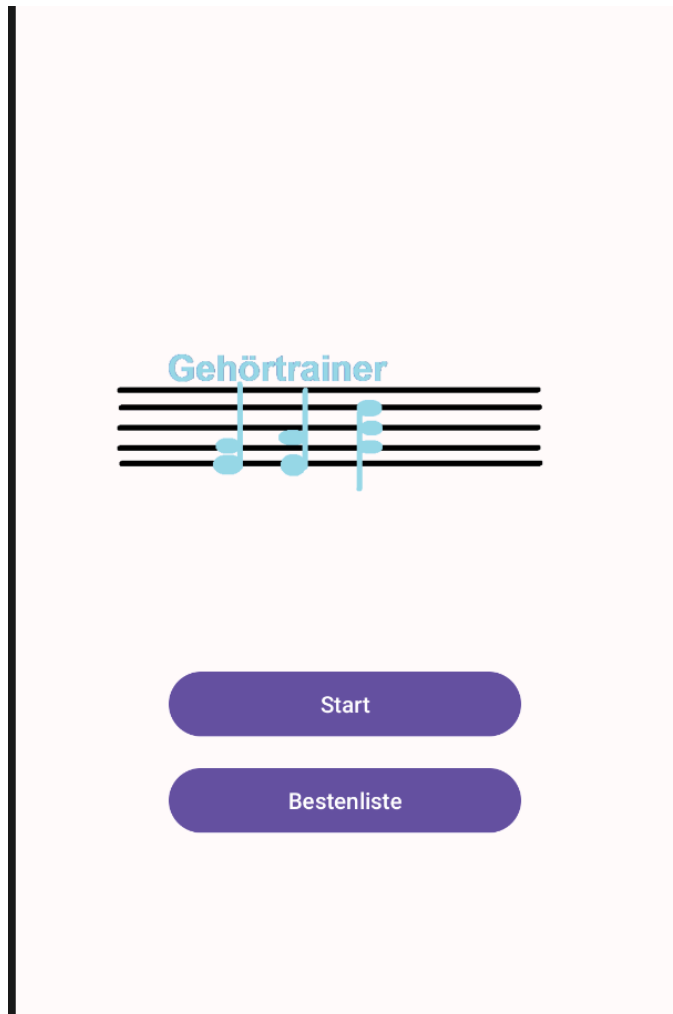
Name	UC-Bestenliste 2 - Austauschen
Scope	Gehörtrainer App
Niveau	Anwenderziel
Actor	Nutzer A+B jeweils mit 1 Gerät (wollen Wertungen austauschen)
Stakeholder	Nutzer A+B (wollen Wertungen austauschen und diese nach denselben Bewertungskriterien erhalten haben)
Vorbedingung	- UC-Bestenliste 1 :: Standardablauf - Bluetooth auf Geräten von Nutzer A+B aktiviert
Mindestzusicherungen	- Erfolg/Misserfolg des Datenaustauschs wird angezeigt - Lokal gespeicherte Daten bleiben konsistent
Zusicherung im Erfolgsfall	- Lokale Bestenliste auf beiden Geräten enthält die Daten von beiden vereinigt
Standardablauf	1. Nutzer A+B wählen 'Austausch' in Bestenliste UI 2. Verbindung wird aufgebaut 3. Daten werden ausgetauscht 4. Bestenliste auf beiden Geräten wird um Daten des jeweils anderen Geräts erweitert 5. Anzeige zum Erfolg des Austauschs 6. Rückkehr zu Hauptmenü
Erweiterungen	1a: Nutzer A oder B wählen 'Abbruch' Button 2a: Nutzer B wählt nicht Austausch - Verbindungsaufbau screen bleibt aktiv mit 'Abbruch' Button verfügbar 3a: Fehler bei Datenübertragung - Fehlermeldung und Rückkehr zum Hauptmenü 4a: inkonsistente Daten erhalten - Fehlermeldung und Rückkehr zum Hauptmenü

2.1 Use Case Diagramm



3 UI Screenshots

3.1 Hauptmenü



3.2 Einstellungen

Einstellungen

ID

Keks

Runden

1

Grundton

fest

Intervall min

1

Intervall max

12

Polyphon

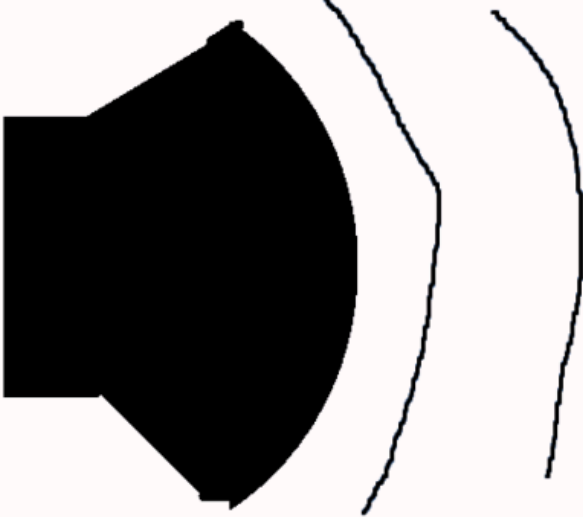
4

Start

Abbrechen

3.3 Raten

Runde 5/5



Intervall 1

2

Intervall 2

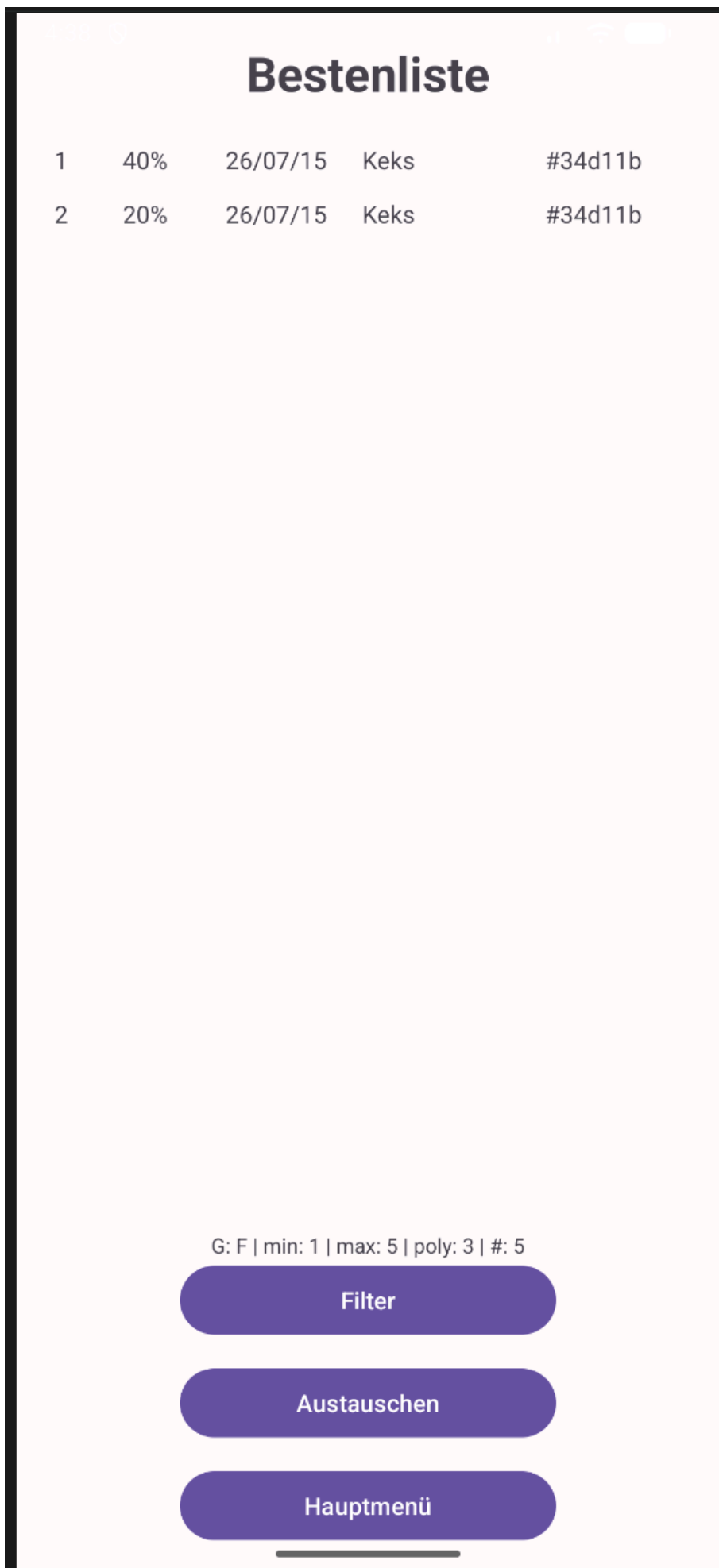
4

Bestätigen

Correct: 1

False: 3

3.4 Ranking



3.5 Filter

Filter

Runden

5

Grundton

fest

Intervall min

1

Intervall max

12

Polyphon

2

Nur eigene ID

Anwenden

Abbrechen

4 Komponenten

Alle Komponenten der Anwendungslogik besitzen ein Interface. Dadurch werden Implementierung und Nutzung voneinander getrennt. Die Benutzeroberfläche arbeitet ausschließlich mit den Interfaces.

Interfaces im Projekt:

ISpiellogik

IAudioDevice

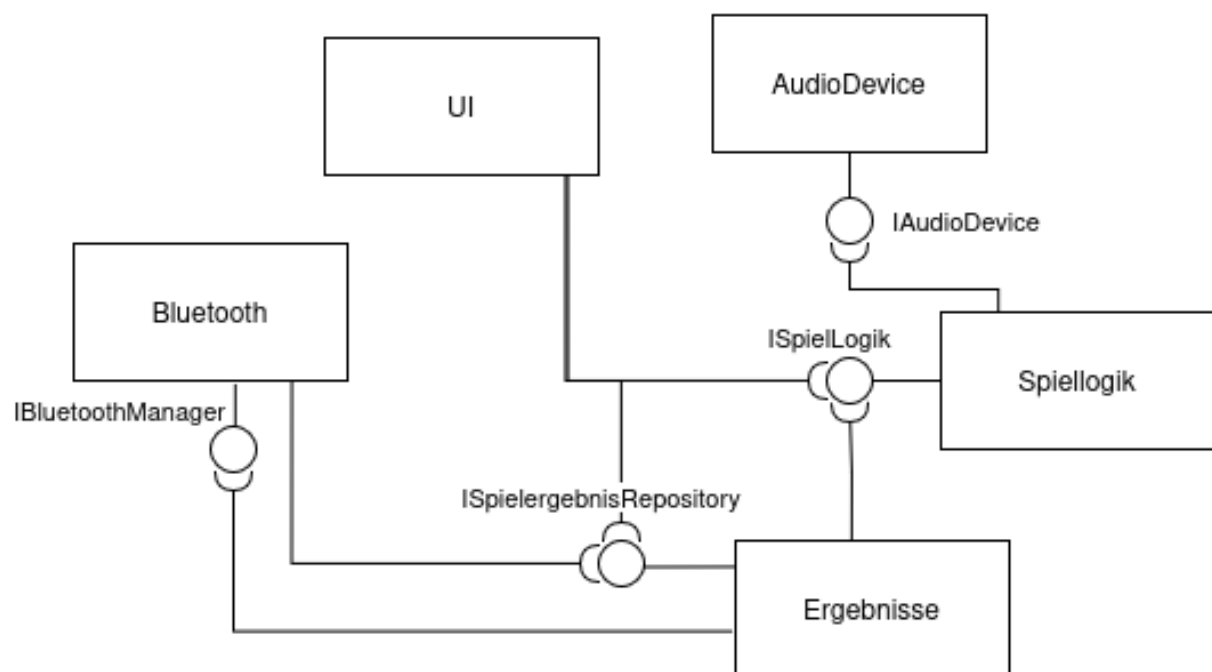
ISpielergebnisRepository

IPreferencesRepository

IBluetoothManager (ohne konkrete Implementierung)

Nachfolgend ist die Strukturierung der Komponenten, wie diese getestet wurden und ihre Beschreibung.

4.1 Komponentendiagramm



4.2 MVC

Die App setzt eine MVC Implementierung um. Dabei ist diese Trennung vorhanden:

4.2.1 Model

SettingsModel - Einstellungen für einen Spielablauf, lokal gespeichert
SpielergebnisModel - Spielergebnisse für die Bestenliste, lokal gespeichert
HighscoreFilter - Filtereinstellungen für die Bestenliste, nicht gespeichert
HighscoreRow - Datencontainer für Visualisierung in Bestenliste

4.2.2 View

Die Activities bilden die View-Schicht der Anwendung.

Aufgrund der Android-Architektur enthalten sie zusätzlich einen kleinen Teil der Steuerungslogik für Navigation und Benutzerinteraktionen. Die eigentliche Fachlogik befindet sich jedoch ausschließlich in den Komponenten der Controller-Schicht.

MainActivity - Hauptmenü
SettingsActivity - Einstellungen festlegen für Rateversuche
GuessActivity - aktives Raten
HighscoreActivity - Bestenliste einsehen
FilterActivity - Filter für Bestenliste einstellen

4.2.3 Controller

Spiellogik - steuert den Spielablauf
AudioDevice - spielt Töne ab
SpielergebnisRepository - steuert das Speichern/Laden von Spielergebnissen
PreferencesRepository - steuert das Speichern/Laden von Spieleinstellungen
InstallationIdManager - steuert das Speichern/Laden einer InstallationID und erzeugt diese
BluetoothManager - (geplant, nicht implementiert) steuert den Verbindungsaufbau, Austausch von Spielergebnissen und beenden der Verbindung
BluetoothSyncController - (nicht vollständig implementiert, für Testzwecke) steuert den Austausch von Spielergebnissen zwischen Geräten

4.3 Spiellogik

Die Komponente Spiellogik verwaltet einen vollständigen Spielablauf.

Aufgaben:

- Erzeugen neuer Intervalle
- Berücksichtigung der Spieleinstellungen
- Bewertung von Benutzereingaben
- Zählen korrekter und falscher Antworten
- Erzeugen eines SpielergebnisModels nach Spielende

4.3.1 ISpiellogik

Dieses Interface bietet die Methoden zur Verwaltung des Spielverlaufs. Die Methoden sind wie folgt:

getCurrentMidiNotes - liefert die numerischen Werte des aktuellen intervalls einschließlich grundton an erster stelle

newInterval - erzeugt die numerischen Werte des nächsten Intervalls zum Abspielen

raten - bewertet die gegebenen intervale und erzeugt das nächste Intervall zum Raten

getCorrectCount - Anzahl der richtig geratenen Runden

getFalseCount - Anzahl der falsch geratenen Runden

getEndergebnis - liefert den finalen Punktestand und Daten über Spieleinstellungen

4.3.2 Spiellogik

Die konkrete Implementierung besitzt keine Abhängigkeit zur Android-Benutzeroberfläche und kann daher isoliert getestet werden.

4.3.3 Tests

Für die Spiellogik wurden diese Testfälle implementiert:

Testfall	Ziel
getCurrentMidiNotes_liefertGenauPolyphonyWerte	Rückgabestruktur prüfen
newInterval_liefertGenauPolyphonyWerte	Intervallerzeugung prüfen
correctUndFalseSindInitialNull	Initialzustand prüfen
raten_mitFalscherAnzahlWirftException	Eingabevalidierung
raten_mitZuKleinemIntervallWirftException	Eingabevalidierung
raten_mitZuGrossemIntervallWirftException	Eingabevalidierung
falscherRateversuch_erhoehtFalseCount	Spiellogik-Verhalten
nachMaximalenRundenWerdenKeineWeiterenPunkteGezaehlt	Rundengrenze
getEndergebnis_uebernimmtSettingsKorrekt	Ergebnisobjekt
getEndergebnis_mitLeeremInstallationIdWirftException	Fehlerbehandlung

4.4 AudioDevice

Spielt Töne während der Rateversuche ab.

4.4.1 IAudioDevice

Dieses Interface bietet die Methoden zum Abspielen von Tönen anhand von Int Werten. Es enthält diese Methoden:

play - spielt eine Liste von Tönen nacheinander und dann gleichzeitig

filterPlayableNotes - Hilfsfunktion für play, um Töne im Wertebereich zu halten

cleanup - zur Freigabe von Ressourcen

4.4.2 SoundpoolDevice

Die konkrete Implementierung verwendet native Android Unterstützung zum Abspielen der Töne: android.media.SoundPool. Die Töne sind dabei Piano Samples unter der Creative Commons License 3.0 heruntergeladen auf <https://freesound.org/people/jobro/packs/2489/>

4.4.3 Tests

Die Klasse SoundpoolDevice verwendet die Android-Komponente SoundPool zum Abspielen von Audiodateien. Da die eigentliche Wiedergabe durch das Android-Betriebssystem erfolgt, wurden keine automatisierten Tests der Wiedergabe durchgeführt. Getestet wurde stattdessen die Hilfsfunktion filterPlayableNotes(), welche Notenwerte auf den unterstützten Bereich 17 bis 63 begrenzt.

4.5 Spielerggebnisse

Speicher für Ergebnisse aus Rateversuchen. Gespeicherte Daten können später über Bluetooth ausgetauscht werden.

4.5.1 ISpielerggebnisRepository

Bietet Methoden zum Speichern/Laden von Spielerggebnissen:

load - lädt Ergebnisse

save - speichert Ergebnisse

4.5.2 SpielerggebnisRepository

Die konkrete Implementierung verwendet android.content.Context zum lokalen Speichern der Daten im JSON Format.

4.5.3 Tests

Da diese Komponente mit AndroidSharedPreferences arbeitet, wurden Integrationstests dafür implementiert. Konkret gibt es diese Testfälle:

Testfall	Ziel
load_ohneEintraege_liefertLeereListe	Leerer Speicher wird korrekt verarbeitet
save_speichertEintrag	Einzelnes Ergebnis speichern
save_mehrereEintraege_werdenAlleGeladen	Mehrere Ergebnisse speichern
save_behaeltReihenfolgeBei	Reihenfolge der Speicherung bleibt erhalten
save_undLoad_erhaltenAlleFelder	JSON-Serialisierung vollständig korrekt

4.6 PreferencesRepository

Speicher für die Spieleinstellungen, um schnelle Neuversuche zu ermöglichen

4.6.1 IPreferencesRepository

Bietet Methoden zum Speichern/Laden von Spieleinstellungen

loadSettings - lädt Spieleinstellungen

saveSettings - speichert Spieleinstellungen

4.6.2 PreferencesRepository

Die konkrete Implementierung verwendet android.content.Context zum lokalen Speichern der Daten.

4.6.3 Tests

Getestet wurden Speichern und Laden von Einstellungen sowie das Laden der Standardwerte bei leerem Speicher. Hierbei handelt es sich auch um Integrationstests, da diese Komponente mit AndroidSharedPreferences arbeitet.

4.7 InstallationIdManager

Erzeugt eine einzigartige ID für die Installation der App, damit beim späteren Austausch von Spielergebnissen über Bluetooth diese eindeutig zuweisbar bleiben.

Die Klasse bleibt nur eine kleine Hilfsklasse, da sie eine sehr einfache und unveränderliche Aufgabe hat: das Speichern einer einzigartigen ID auf dem Gerät.

4.7.1 Tests

Auch diese Komponente arbeitet mit AndroidSharedPreferences und dafür habe ich diese Testfälle als Integrationstests:

Testfall	Ziel
getInstallationId_createsId_whenNoneExists	Erzeugung einer neuen InstallationID bei leerem Speicher
getInstallationId_returnsSameId_onRepeatedCalls	Wiederholtes Laden liefert dieselbe InstallationID
getInstallationId_persistsId_betweenInstances	Die InstallationID bleibt auch nach Neuerzeugung des Managers erhalten

4.8 Bluetooth

Verbindung zwischen zwei Geräten für den Austausch von Spielergebnissen.

Es wird angenommen, dass die Bluetooth Verbindung auf den Geräten funktioniert.

Die Komponente Bluetooth wurde nicht implementiert, da die restlichen Komponenten bereits das gesamte verfügbare Zeitbudget aufgebraucht haben.

4.8.1 IBluetoothManager

Die Bluetooth-Komponente wurde nicht implementiert. Stattdessen wurde die Schnittstelle IBluetoothManager mittels Mockito gemockt. Dadurch konnte das Verhalten eines zukünftigen Bluetooth-Controllers getestet werden, einschließlich korrekter Methodenaufrufe, Aufrufreihenfolge und Fehlerfällen, ohne auf reale Bluetooth-Hardware angewiesen zu sein.

Das Interface biete diese Methoden an:

connect - Verbindung aufbauen

exchangeResults - Spielergebnisse austauschen

disconnect - Verbindung beenden

4.8.2 BluetoothSyncController

Unvollständig implementiert für Testzwecke - ein simulierter Austausch von Spielergebnissen zwischen Geräten

4.8.3 Tests

Für die Tests wurde mockito-kotlin:5.4.0 verwendet, um Integrationstests zu schreiben.

Test	Ziel
sync_ruft_alle_bluetooth_methoden_auf	Alle für Verbindungsaufbau relevanten Methoden aufgerufen
sync_ruft_methoden_in_richtiger_reihenfolge_auf	Alle relevanten Methoden in richtiger Reihenfolge aufgerufen
sync_uebergibt_ergebnisse	Verbindungs austausch übergibt Daten
sync_wirft_fehler_wenn_connect_fehlgeschlagen	Fehlerabdeckung

4.9 UI

Die **HighscoreActivity** verwendet ein RecyclerView, um die Highscores als Listenelemente darzustellen. Dafür gibt es einen **HighscoreAdapter**.

Für die **SettingsActivity** sind die intervalMin und intervalMax Slider so verbunden, dass min niemals $\geq$ max wird und umgekehrt.

In der **GuessActivity** wird die Slideranzahl an die Polyphonie und möglichen Intervallwerte entsprechend der Einstellungen angepasst.

4.9.1 Tests

Für End-to-End Tests mit der UI habe ich das Espresso Framework verwendet. Die Testfälle waren dabei wie folgt:

Test	Ziel
MainActivityNavigationTest	Die Buttons im Hauptmenü führen jeweils zu anderen Views
SettingsActivityTest	Aufbau der Einstellungsansicht prüfen
FilterActivityTest	Navigation und Rückkehr ohne Filteränderung prüfen -> beschreibt einen vollständigen Use-Case: Bestenliste einsehen

5 Rückblick auf das Projekt

Das Ziel, einen Hörtrainer zu schaffen, der einfache bis sehr komplexe Übungen erlaubt, wurde erreicht.

Von den geplanten Features wurden alle bis auf den Austausch von Spielergebnissen implementiert. Dies ist eine gut mögliche Erweiterung für einen späteren Zeitpunkt. Die Code-Struktur für einfaches Einbauen ist gegeben.

Unerwartete Probleme, die etwa 12 Stunden insgesamt aus der Entwicklungszeit gekostet haben, hingen mit JDK Einstellungen auf der Entwicklermaschine zusammen und bleiben bis heute ungelöst. Zwar konnte das Projekt mit gradle problemlos gebaut werden und auf dem Emulator in Android Studio ausgeführt werden, aber die Tests ließen sich nicht kompilieren und lieferten dabei auch nur nichts-sagende Exceptions.

Stattdessen wurde auf einen zweiten PC für die Ausführung der Tests ausgewichen. Auch dort gab es zunächst Probleme, weil gradle mit Umlauten im Windows-Benutzernamen nicht umgehen konnte. Auch hier waren die geworfenen Exceptions von geringem Aussagewert, aber ich stieß zufällig beim Troubleshooting auf einen Hinweis über Umlaute. Anschließend konnten die Tests auf dem zweiten PC ausgeführt werden.

Es wurde keine Zeit für optische Anpassungen verwendet. So könnte noch ein style theme erstellt werden, Elemente auf Views größer/kleiner gemacht werden und bessere Bilddateien für Logo und Abspielknopf verwendet werden.

6 Repository und Videopräsentation

Das Repository für die App ist unter

https://gitlab.rz.htw-berlin.de/s0587471/26ss_gehoertrainer

Die Videopräsentation ist in der HTW-Mediathek

<https://mediathek.htw-berlin.de/video/mobile-betriebssysteme-und-netzwerke-abschlusspräsentation-gehrtrainer/8d5fa2138b13dd93448a3a63bc0e8984>

7 Beschreibung Anwendungsablauf

An jeder Stelle in der App führt ein Klick auf Android-Back zurück zum Hauptmenü.

7.1 Hauptmenü

“Starten” führt zu den Einstellung für die Raterunde.

“Bestenliste” führt zu einer Auflistung der Spielergebnisse.

7.2 Einstellungen

Vorherige Einstellungen bleiben bestehen, damit direkt mit denselben Einstellungen neu geraten werden kann.

Grundton fest/variabel: wenn variabel hat der Grundton in jeder Runde einen zufälligen Wert im Wertebereich [Grundton der App - 11, Grundton der App +11]. Wenn fest bleibt der Grundton auf dem Grundton der App. Der Grundton der App ist aktuell C3.

Die Slider für min und max Intervall bleiben automatisch konsistent, d.h. min kann nicht gleich oder mehr als max sein.

Wenn für die eingestellte Polyphonie mehr Intervallgröße nötig wäre, weist die UI mit einem Toast darauf hin.

7.3 Raten

Ein Klick auf das dargestellte Lautsprechersymbol spielt das Intervall bzw den Akkord ab.

Es werden so viele Slider dargestellt, wie es Töne im Akkord minus Grundton gibt. Die Slider haben nur gültige Werte, die tatsächlich auftauchen könnten. Die Reihenfolge der Werte auf den Slidern ist für die Bewertung nicht relevant; d.h. niedrigster+mittlerer+höchster ist genauso richtig wie höchster+niedrigster+mittlerer oder jede beliebige andere Ordnung.

Mit Klick auf “Bestätigen” wird der Versuch ausgewertet und die nächste Runde fängt an. Wenn in der letzten Runde bestätigt wird, geht die App zurück zum Hauptmenü. Das Ergebnis des Spiels wird dann in der Bestenliste gespeichert.

7.4 Bestenliste

Zeigt eine Auflistung nach bestem prozentualen Ergebnis sortiert. Beim öffnen wird immer kein Filter angewandt.

Oberhalb des Filterbuttons ist immer zu sehen, welcher Filter aktuell verwendet wird.

Klick auf "Filter" öffnet die FilterActivity.

Klick auf "Austauschen" ist für Bluetooth Austausch konzipiert und nicht implementiert.

Klick auf "Hauptmenü" führt zurück zum Hauptmenü.

7.5 Filter

Hier kann für spezifische Spieleinstellungen ein Filter festgelegt werden, der auf die Darstellung in Bestenliste angewandt wird.

Die Inputs sind gleich mit denen der Spieleinstellungen, aber ihre Werte nicht persistent. D.h. bei einem erneuten Aufruf der Bestenliste vom Hauptmenü aus ist kein Filter eingestellt. Der Filter "nur eigene ID" funktioniert, aber hat keine Relevanz, solange der Austausch über Bluetooth nicht implementiert ist.

Ein Klick auf "Anwenden" führt zur Bestenliste und wendet dort den eingestellten Filter an.

Ein Klick auf "Abbrechen" führt zur Bestenliste und führt keine Änderung an den Filtereinstellungen aus.